

Linux Filesystem

Prathiksha Patil
261100690008

Exercise 1 – Identify Your Linux Filesystem

Explore the storage configuration of your Linux system.

Determine:

1. The storage devices available.

Storage devices available: sda (20G disk) and sr0 (a 1024M optical/CD-ROM drive, TYPE=rom).

2. The partitions available on each device.

Partitions on sda: sda1 (18.9G), sda2 (1K — this is an *extended* partition, a container that doesn't hold data itself), and sda5 (1.1G — a logical partition inside the extended one). That's the classic MBR pattern where you needed >4 primary partitions.

3. The filesystem type used by each partition.

Filesystem type per partition: sda1 → ext4; sda2 → none (it's just the extended partition container, no filesystem of its own); sda5 → swap.

4. The device/partition on which '/' is located..

Device / is on: /dev/sda1.

5. The filesystem type used for '/'.

Filesystem type for /: ext4

6. The total, used, and available storage space.

Total/used/available space for /: from df -h → 19G total, 17G used, 821M available, 96% used.

```
(prathiksha@kali)-[~]
$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda   8:0    0  20G  0 disk
├─sda1 8:1    0 18.9G  0 part /
├─sda2 8:2    0    1K  0 part 
├─sda5 8:5    0   1.1G  0 part [SWAP]
sr0   11:0    1 1024M  0 rom

(prathiksha@kali)-[~]
$ lsblk -f
NAME FSTYPE FSVER LABEL UUID                                 FSAVAIL FSUSE% MOUNTPOINTS
sda
├─sda1 ext4   1.0             89648abf-dd17-46b6-984f-7f5e87d56a92  820.2M   90% /
├─sda2
├─sda5 swap    1              f59d8ea7-2a0e-400b-8b47-33f45ed0ef44          [SWAP]
sr0

(prathiksha@kali)-[~]
$ df -T
Filesystem      Type      1K-blocks    Used Available Use% Mounted on
udev            devtmpfs   972956         0    972956    0% /dev
tmpfs           tmpfs      201688        1028    200660    1% /run
/dev/sda1       ext4      19353424    17505084    839908    96% /
tmpfs           tmpfs      1008428         4    1008424    1% /dev/shm
none            tmpfs       1024         0       1024    0% /run/credentials/systemd-journald.service
tmpfs           tmpfs      1008432         8    1008424    1% /tmp
none            tmpfs       1024         0       1024    0% /run/credentials/getty@tty1.service
tmpfs           tmpfs      201684        112    201572    1% /run/user/1000

(prathiksha@kali)-[~]
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            951M   0    951M   0% /dev
tmpfs           197M 1020K  196M   1% /run
/dev/sda1       19G   17G   821M  96% /
tmpfs           985M  4.0K  985M   1% /dev/shm
none            1.0M   0    1.0M   0% /run/credentials/systemd-journald.service
tmpfs           985M  8.0K  985M   1% /tmp
none            1.0M   0    1.0M   0% /run/credentials/getty@tty1.service
tmpfs           197M 112K  197M   1% /run/user/1000
```

Record your findings in a table.

Device/Partition	Filesystem Type	Size	Mount Point
------------------	-----------------	------	-------------

/dev/sda1	ext4	18.9G	/
/dev/sda2	— (extended)	1K	–
/dev/sda5	swap	1.1G	– (swap)
/dev/sr0	—	1024M	— (optical, drive)

Exercise 2 – Disk → Partition → Filesystem → Mount Point

Using the information obtained in Exercise 1, draw the storage organization of your own Linux system. Explain in 2–3 sentences the relationship between these four components.

sda (20G disk)

```

|
+-- sda1 (18.9G partition)
|   |
|   +-- ext4 (filesystem)
|       |
|       +-- / (mount point)
|
+-- sda2 (1K extended partition — container only, no filesystem)
|
|   +-- sda5 (1.1G logical partition)
|       |
|       +-- swap (filesystem)
|           |
|           +-- (no mount point — swap space is not mounted into the directory tree)

```

The physical disk `sda` is divided into partitions (`sda1`, `sda2`, `sda5`), each of which is a distinct storage region that can be managed independently. Each partition is formatted with a filesystem — `sda1` uses `ext4` and `sda5` uses `swap` — which defines how data is structured, stored, and retrieved on that partition. A filesystem is then attached to the Linux directory tree at a mount point (in this case, `sda1`'s `ext4` filesystem is mounted at `/`), making its contents accessible as ordinary files and folders; `swap`, by contrast, isn't mounted anywhere since it's used directly by the kernel for memory paging rather than for storing files.

Exercise 3 – Explore the Root Filesystem

Start from your current location.

1. Determine your current working directory.
2. Navigate to the root directory.
3. Display the contents of the root directory.
4. Identify all directories directly available under `/`.
5. Determine whether hidden entries are present.

```

(prathiksha@kali)~$ pwd
/home/prathiksha

(prathiksha@kali)~$ cd /

(prathiksha@kali)/$ pwd
/

(prathiksha@kali)/$ ls
bin boot dev etc home initrd.img initrd.img.old lib lib32 lib64 lost+found media mnt opt proc root run sbin srv sys tmp usr var vmlinuz vmlinuz.old

(prathiksha@kali)/$ ls -a
. .. bin boot .cache dev etc home initrd.img initrd.img.old lib lib32 lib64 lost+found media mnt opt proc root run sbin srv sys tmp usr var vmlinuz vmlinuz.old

```

Exercise 4 – Linux Filesystem Hierarchy Investigation

For each location:

1. Examine its contents.
2. Identify at least three entries, where available.
3. Determine the likely purpose of the directory.

Directory	Example Contents	Purpose
/home	., ., prathiksha (mode drwx-----)	Holds personal directories for each regular user account; each user's folder is locked to that user only
/root	.bashrc, .zshrc, .ssh, .msf4, .mariadb_history	Home directory for the root (superuser) account, kept separate from /home and restricted (mode 700)
/etc	passwd, shadow, hosts, fstab, sudoers, ssh/	System-wide configuration files for the OS and all installed applications/services
/var	log, cache, spool, mail, www, tmp	Variable data that changes during operation — logs, caches, mail spool, web files
/var/log	boot.log, wtmp, btmp, Xorg.0.log, journal/, apt/	Stores system, boot, login, and application logs used for troubleshooting and auditing
/tmp	.X11-unix, .ICE-unix, config-err-mXpN8Q, systemd-private-* dirs	World-writable temporary storage for session/runtime files; sticky bit set; cleared on reboot
/usr	bin, lib, share, local, include	Holds installed user programs, libraries, and shared documentation/data
/boot	vmlinuz-6.19.14+kali-amd64, initrd.img-6.19.14+kali-amd64, grub/	Contains the kernel image, initial ramdisk, and bootloader files needed to start the system
/dev	sda, sda1, sda2, sda5, null, zero, tty1, sr0	Device files representing hardware and virtual devices, accessed like normal files
/proc	cpuinfo, meminfo, self, numbered PID folders (e.g. 1, 16956)	Virtual, kernel-generated filesystem exposing live process and system information; nothing stored on disk

Exercise 5 – Find Your Home Directory

Without assuming its location:

1. Determine your username.
2. Determine your home directory.
3. Navigate to your home directory.
4. Display its absolute path.
5. List its normal contents.
6. List all contents, including hidden files.

```

(prathiksha@kali)-[/]
$ whoami
prathiksha

(prathiksha@kali)-[/]
$ echo $HOME
/home/prathiksha

(prathiksha@kali)-[/]
$ cd ~

(prathiksha@kali)-[~]
$ pwd
/home/prathiksha

(prathiksha@kali)-[~]
$ ls
a.txt b.txt client.py crypto_4 crypto_lab Desktop Documents Downloads grep_lab

(prathiksha@kali)-[~]
$ ls -la
total 248
drwx----- 29 prathiksha prathiksha 4096 Aug 31 22:33 .
drwxr-xr-x  3 root      root      4096 Aug 11 10:07 ..
-rw-rw-r--  1 prathiksha prathiksha   0 Aug 20 20:50 a.txt
-rw-----  1 prathiksha prathiksha  257 Aug 20 21:01 .bash_history
-rw-r--r--  1 prathiksha prathiksha  220 Aug 11 10:07 .bash_logout
-rw-r--r--  1 prathiksha prathiksha 5578 Aug 11 10:07 .bashrc
-rw-r--r--  1 prathiksha prathiksha 3526 Aug 11 10:07 .bashrc.original
-rw-rw-r--  1 prathiksha prathiksha   0 Aug 20 20:50 b.txt
drwxrwxr-x 12 prathiksha prathiksha 4096 Aug 16 22:49 .cache
-rw-rw-r--  1 prathiksha prathiksha  274 Aug 24 23:37 client.py
drwxr-xr-x 18 prathiksha prathiksha 4096 Aug 15 17:43 .config
drwxrwxr-x  2 prathiksha prathiksha 4096 Aug 30 15:24 crypto_4

```

Exercise 6 – Hidden Files Investigation

Examine your home directory.

Compare:

ls

ls -a

Identify at least 2 hidden files/directories.

- .bash_history — stores your shell command history
- .ssh — hidden directory holding SSH keys and configuration

```

(prathiksha@kali)-[~]
$ cd ~

(prathiksha@kali)-[~]
$ ls
a.txt b.txt client.py crypto_4 crypto_lab Desktop Documents Downloads grep_lab lab1 Music permission_test Pictures

(prathiksha@kali)-[~]
$ ls -a
.          .bashrc.original  crypto_lab  .face.icon  lab1        Pictures  Projects  testfolder
..         b.txt            Desktop    .gnupg      .local      practice  Public    test.txt
a.txt     .cache           .dmrc      grep_lab    .mozilla    practice11 .python_history .vboxclient-d
.bash_history client.py        Documents  .hash.txt.swp .msf4       practice22 .ssh       .vboxclient-d
.bash_logout .config          Downloads  .ICEauthority Music       practice33 .sudo_as_admin_successful .vboxclient-d
.bashrc    crypto_4         .face      .java       permission_test .profile  Templates .vboxclient-d

```

1. How does Linux identify a hidden file?

Any file or directory whose name starts with a dot (.) is treated as hidden — confirmed clearly here, since every extra entry `ls -a` revealed begins with ..

2. Why does a normal 'ls' not display these files?

ls filters out dot-prefixed entries by default purely for readability, so users aren't overwhelmed by config/cache clutter — it's a display convention, not a security control.

3. What do '.' and '..' mean?

. is the current directory itself; .. is its parent directory — both appear as the first two entries in every `ls -a` listing, as seen here.

4. Can a hidden file still be accessed if its name is known?

Yes — e.g. `cat .bash_history` or `cd .ssh` works exactly the same as any normal file/directory; the dot prefix has zero effect on actual access, only on default visibility.

5. From a cybersecurity perspective, why should hidden files not automatically be considered safe or malicious?

This listing itself makes the point well: it mixes completely mundane hidden files (.bashrc, .profile, .bash_history) with sensitive ones (.ssh, .gnupg) and an incidental artifact (.hash.txt.swp — possibly leftover from editing something named hash.txt, which could be innocuous or could be evidence of password-cracking activity depending on context). The naming convention alone tells you nothing about intent or risk; an investigator has to look at each hidden item's actual content, permissions, and context rather than assuming "hidden = suspicious" or "hidden = safe."

Exercise 7 – Build a Small Filesystem Hierarchy

Explore: mkdir

Hint: you can create this hierarchy in a single command

Create the following files:

cyber_lab/evidence/network/traffic.txt

cyber_lab/evidence/system/processes.txt

cyber_lab/logs/security.log

cyber_lab/scripts/check.sh

cyber_lab/reports/investigation.txt

Use appropriate Linux commands to create the structure.

Verify that the required hierarchy has been created successfully.

Hint: tree

Explore options

```
(prathiksha@kali)~$ cd ~
(prathiksha@kali)~$ mkdir -p cyber_lab/evidence/{network,system} cyber_lab/{logs,scripts,reports}
(prathiksha@kali)~$ touch cyber_lab/evidence/network/traffic.txt
(prathiksha@kali)~$ touch cyber_lab/evidence/system/processes.txt
(prathiksha@kali)~$ touch cyber_lab/logs/security.log
(prathiksha@kali)~$ touch cyber_lab/scripts/check.sh
(prathiksha@kali)~$ touch cyber_lab/reports/investigation.txt
(prathiksha@kali)~$ tree cyber_lab
cyber_lab
├── evidence
│   ├── network
│   │   └── traffic.txt
│   └── system
│       └── processes.txt
├── logs
│   └── security.log
├── reports
│   └── investigation.txt
└── scripts
    └── check.sh
7 directories, 5 files
```

Exercise 8 – Absolute and Relative Paths

Assume your current directory is:

~/cyber_lab

Perform the following:

1. Navigate to `evidence/network` using a relative path.
2. Navigate to `/etc` using an absolute path.
3. Return to your home directory using the shortest possible command.
4. Navigate to `cyber\_lab/reports`.
5. Move one directory upward.
6. Display the current directory after every operation.

```

(prathiksha@kali)~]
$ cd ~/cyber_lab

(prathiksha@kali)~/cyber_lab]
$ pwd
/home/prathiksha/cyber_lab

(prathiksha@kali)~/cyber_lab]
$ cd evidence/network

(prathiksha@kali)~/cyber_lab/evidence/network]
$ pwd
/home/prathiksha/cyber_lab/evidence/network

(prathiksha@kali)~/cyber_lab/evidence/network]
$ cd /etc

(prathiksha@kali)~/etc]
$ pwd
/etc

(prathiksha@kali)~/etc]
$ cd

(prathiksha@kali)~]
$ pwd
/home/prathiksha

(prathiksha@kali)~]
$ cd cyber_lab/reports

(prathiksha@kali)~/cyber_lab/reports]
$ pwd
/home/prathiksha/cyber_lab/reports

(prathiksha@kali)~/cyber_lab/reports]
$ cd ..

(prathiksha@kali)~/cyber_lab]
$ pwd
/home/prathiksha/cyber_lab

```

* What is an absolute path?

A path that starts from the filesystem root / and fully specifies a location regardless of your current directory — e.g. /home/prathiksha/cyber_lab/evidence/network.

* What is a relative path?

A path interpreted relative to your current working directory — e.g. evidence/network only resolves correctly if you're already inside cyber_lab.

* What is the difference between `/` and `~`?

/ is the root of the entire filesystem, the top of the whole directory tree. ~ is shorthand for the current user's home directory (/home/prathiksha in your case) — it's a convenience alias, not a real filesystem location.

Exercise 9 – Identify File Types

Investigate the following:

/etc/passwd

/etc

/bin/bash

/dev/null

/proc

Also investigate the files created in Exercise 7.

file <filename>

ls -l <filename>

Classify each object.

```

(prathiksha@kali)~[/cyber_lab]
$ file /etc/passwd
/etc/passwd: ASCII text

(prathiksha@kali)~[/cyber_lab]
$ file /etc
/etc: directory

(prathiksha@kali)~[/cyber_lab]
$ file /bin/bash
/bin/bash: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=e1983fd58f88f72cd98b92c28692a9eef51d8da1, for GNU/Linux 3.2.0, stripped

(prathiksha@kali)~[/cyber_lab]
$ file /dev/null
/dev/null: character special (1/3)

(prathiksha@kali)~[/cyber_lab]
$ file /proc
/proc: directory

(prathiksha@kali)~[/cyber_lab]
$ ls -l /etc/passwd
-rw-r--r-- 1 root root 3281 Aug 11 10:12 /etc/passwd

(prathiksha@kali)~[/cyber_lab]
$ ls -l -d /etc
drwxr-xr-x 189 root root 12288 Aug 31 22:32 /etc

(prathiksha@kali)~[/cyber_lab]
$ ls -l /bin/bash
-rwxr-xr-x 1 root root 1384752 May 4 00:58 /bin/bash

(prathiksha@kali)~[/cyber_lab]
$ ls -l /dev/null
crw-rw-rw- 1 root root 1, 3 Aug 31 22:32 /dev/null

(prathiksha@kali)~[/cyber_lab]
$ ls -l -d /proc
dr-xr-xr-x 221 root root 0 Aug 31 22:32 /proc

(prathiksha@kali)~[/cyber_lab]
$ cd ~/cyber_lab

```

Object	Type
/etc/passwd	Regular text file (ASCII text — user account database)
/etc	Directory
/bin/bash	Regular file — ELF executable binary (or symlink to /usr/bin/bash on modern Kali)
/dev/null	Character special device file
/proc	Directory — root of the virtual/pseudo procfs filesystem
evidence/network/traffic.txt	Regular (empty) file
evidence/system/processes.txt	Regular (empty) file
logs/security.log	Regular (empty) file
scripts/check.sh	Regular (empty) file — would become an executable shell script once given a shebang and +x permission
reports/investigation.txt	Regular file (no longer empty after Exercise 10, once you add text to it)

Exercise 10 – Examine File Metadata

Use:

~/cyber_lab/reports/investigation.txt

Add a small amount of text to the file.

Examine it using:

ls -l

stat investigation.txt

Identify:

- * Filename
- * File size
- * File type
- * Owner
- * Group
- * Inode number
- * Access time
- * Modification time
- * Change time

```
(prathiksha@kali)-[~/cyber_lab]
$ echo "Investigation notes: initial filesystem review" >> ~/cyber_lab/reports/investigation.txt

cd ~/cyber_lab/reports
ls -l investigation.txt
stat investigation.txt
-rw-rw-r-- 1 prathiksha prathiksha 47 Aug 31 23:56 investigation.txt
  File: investigation.txt
  Size: 47          Blocks: 8          IO Block: 4096   regular file
Device: 8,1      Inode: 1080079    Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/prathiksha)   Gid: ( 1000/prathiksha)
Access: 2026-08-31 23:29:35.605396143 +0530
Modify: 2026-08-31 23:56:46.526960503 +0530
Change: 2026-08-31 23:56:46.526960503 +0530
 Birth: 2026-08-31 23:29:35.605396143 +0530
```

Filename: investigation.txt

File size: 47 bytes

File type: regular file

Owner: prathiksha

Group: prathiksha

Inode number: 1008079

Access time (atime): 2026-08-31 23:29:35.605396143 +0530

Modification time (mtime): 2026-08-31 23:56:46.526960503 +0530

Change time (ctime): 2026-08-31 23:56:46.526960503 +0530

What is metadata?

Data describing a file — its size, owner, group, permissions, timestamps, and inode — rather than the file's actual content.

Is metadata the same as file content?

No. You can change metadata (e.g. chmod, chown, rename) without touching content, and you can edit content without directly touching most metadata (though mtime/ctime still update automatically).

Which timestamp changes when file contents are modified?

mtime (modification time) changes directly. ctime also updates as a side effect (since the inode's size/mtime metadata changed), but atime only changes on read access, not on write.

Exercise 11 – Explore `/etc`

Investigate:

/etc

...

Locate:

passwd

group

hostname

hosts

Note: Without modifying any system files, examine their contents where permitted.

```
(prathiksha@kali)~[~/cyber_lab/reports]
$ ls -l /etc/passwd /etc/group /etc/hostname /etc/hosts
cat /etc/passwd
cat /etc/group
cat /etc/hostname
cat /etc/hosts
-rw-r--r-- 1 root root 1455 Aug 15 09:20 /etc/group
-rw-r--r-- 1 root root 5 Aug 11 09:17 /etc/hostname
-rw-r--r-- 1 root root 199 Aug 11 09:17 /etc/hosts
-rw-r--r-- 1 root root 3281 Aug 11 10:12 /etc/passwd
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
```

What kind of information is primarily stored under /etc?

Host-wide, static configuration files controlling the OS and every installed application/service — no user data or binaries.

What does /etc/passwd contain?

One line per user account: username, placeholder password field (x, since real hashes live in /etc/shadow), UID, GID, GECOS comment, home directory, login shell.

What does /etc/hostname contain?

The system's configured hostname (e.g. kali), a single line.

Why is /etc security-sensitive?

It defines user accounts, authentication rules (/etc/shadow, /etc/sudoers), network configuration, and service behavior. Unauthorized changes here can create backdoor accounts, escalate privileges, redirect network traffic (/etc/hosts), or weaken authentication — as your own `ls -la /etc` output showed, most of it is root-owned and root-writable only.

Exercise 12 – Explore Linux Log Storage

Investigate:

/var/log

...

```
(prathiksha@kali)~[~/cyber_lab/reports]
$ ls -la /var/log
ls -lt /var/log | head # sorted by most recently modified
sudo cat /var/log/auth.log 2>/dev/null | tail -20
cat /var/log/dpkg.log | tail -20
total 1952
drwxr-xr-x 22 root root 4096 Aug 31 23:20 .
drwxr-xr-x 12 root root 4096 Aug 11 10:14 ..
-rw-r--r-- 1 root root 86693 Aug 11 10:12 alternatives
drwxr-x 2 root adm 4096 Aug 22 18:20 apache2
drwxr-xr-x 2 root root 4096 Aug 20 20:56 apt
-rw-r--r-- 1 root root 0 Aug 31 23:20 boot.log
```

Identify

1. Available log files

alternatives — file, 86,693 bytes

boot.log — file

apt — this is a directory, not a file

2. Available log directories

apache2/

apt/

3. Recently modified entries

The `ls -lt /var/log | head` output is sorted by most recently modified. From the visible entries:

```
boot.log — Aug 31 23:20
. — Aug 31 23:14
.. — Aug 31 23:14
alternatives — Aug 11 10:12
apache2/ — Aug 20 20:56
apt/ — Aug 20 20:56
```

4. Entries you can read

`boot.log` appears readable (`-rw-r-----`)

`alternatives` appears readable (`-rw-r--r--`)

The `apache2` and `apt` directories are accessible/listable based on their permissions (`drwxr-xr-x`).

5. Entries you cannot read as a normal user

The screenshot shows most entries owned by `root:root`. For files with permissions such as `-rw-r-----`, a normal user who is not in the owning group cannot read them.

Answers

1. Why does Linux maintain logs?

Linux maintains logs to record system, application, authentication, and security-related events. They help administrators understand what happened on the system.

2. Why are logs useful during incident investigation?

Logs provide evidence about events such as logins, failed authentication attempts, service activity, errors, and system changes. Investigators can use timestamps and event details to reconstruct what happened.

3. Why might access to some logs be restricted?

Some logs contain sensitive information, such as authentication details, user activity, or system-security information. Restricting access prevents unauthorized users from viewing or modifying this information and helps preserve the integrity of the logs.

Exercise 13 – Investigate `/tmp`

Explore `/tmp` without modifying files belonging to other users.

Determine:

1. Whether files/directories currently exist.

Yes. `/tmp` contains several files and directories used for temporary files, sockets, and application/session data.

2. Whether hidden files exist.

Yes. Hidden entries are present, such as `.X11-unix`, `.ICE-unix`, and other dot-prefixed entries.

3. Who owns some of the entries.

The entries may be owned by `root` or by the current user, depending on which process created them.

4. Which entries were modified recently.

The command `ls -lt /tmp` displays entries in order of modification time, with the most recently modified entries appearing first.

```
(prathiksha@kali)~$ ls -la /tmp
total 12
drwxrwxrwt 13 root root 340 Sep 1 22:55
drwxr-xr-x 19 root root 4096 Aug 11 10:01
-rw- 1 prathiksha prathiksha 0 Sep 1 22:55 config-err-vyCvjQ
drwxrwxrwt 2 root root 40 Sep 1 22:55 font-unix
drwxrwxrwt 2 root root 60 Sep 1 22:55 ICE-unix
srw-rw-rw- 1 root root 0 Sep 1 22:54 iprt-localipc-DRMIpcServer
drwx- 3 root root 60 Sep 1 22:55 systemd-private-5b3c76ac0c3d443db5126f21f584e22b-colord.service-KVLaCz
drwx- 3 root root 60 Sep 1 22:55 systemd-private-5b3c76ac0c3d443db5126f21f584e22b-ModemManager.service-f6vMrt
drwx- 3 root root 60 Sep 1 22:55 systemd-private-5b3c76ac0c3d443db5126f21f584e22b-pcsd.service-i7cV04
drwx- 3 root root 60 Sep 1 22:54 systemd-private-5b3c76ac0c3d443db5126f21f584e22b-polkit.service-NDomju
drwx- 3 root root 60 Sep 1 22:55 systemd-private-5b3c76ac0c3d443db5126f21f584e22b-systemd-logind.service-9WvMrt
drwxrwxrwt 2 root root 40 Sep 1 22:54 VMwareDnD
-r--r--r-- 1 root root 11 Sep 1 22:54 X0-lock
drwxrwxrwt 2 root root 60 Sep 1 22:54 X11-unix
-rw- 1 prathiksha prathiksha 398 Sep 1 22:55 xfsm-ICE-ZC19U3
drwxrwxrwt 2 root root 40 Sep 1 22:54 XIM-unix

(prathiksha@kali)~$ ls -la /tmp
total 12
drwxrwxrwt 12 root root 320 Sep 1 22:56
drwxr-xr-x 19 root root 4096 Aug 11 10:01
-rw- 1 prathiksha prathiksha 0 Sep 1 22:55 config-err-vyCvjQ
drwxrwxrwt 2 root root 40 Sep 1 22:54 font-unix
drwxrwxrwt 2 root root 60 Sep 1 22:55 ICE-unix
srw-rw-rw- 1 root root 0 Sep 1 22:54 iprt-localipc-DRMIpcServer
drwx- 3 root root 60 Sep 1 22:55 systemd-private-5b3c76ac0c3d443db5126f21f584e22b-colord.service-KVLaCz
drwx- 3 root root 60 Sep 1 22:54 systemd-private-5b3c76ac0c3d443db5126f21f584e22b-ModemManager.service-f6vMrt
drwx- 3 root root 60 Sep 1 22:54 systemd-private-5b3c76ac0c3d443db5126f21f584e22b-polkit.service-NDomju
drwx- 3 root root 60 Sep 1 22:55 systemd-private-5b3c76ac0c3d443db5126f21f584e22b-systemd-logind.service-9WvMrt
drwxrwxrwt 2 root root 40 Sep 1 22:54 VMwareDnD
-r--r--r-- 1 root root 11 Sep 1 22:54 X0-lock
drwxrwxrwt 2 root root 60 Sep 1 22:54 X11-unix
-rw- 1 prathiksha prathiksha 398 Sep 1 22:55 xfsm-ICE-ZC19U3
drwxrwxrwt 2 root root 40 Sep 1 22:54 XIM-unix

(prathiksha@kali)~$ ls -lt /tmp
total 0
drwx- 3 root root 60 Sep 1 22:55 systemd-private-5b3c76ac0c3d443db5126f21f584e22b-colord.service-KVLaCz
drwx- 3 root root 60 Sep 1 22:55 systemd-private-5b3c76ac0c3d443db5126f21f584e22b-upower.service-VQpwpD
-rw- 1 prathiksha prathiksha 0 Sep 1 22:55 config-err-vyCvjQ
drwx- 3 root root 60 Sep 1 22:54 systemd-private-5b3c76ac0c3d443db5126f21f584e22b-ModemManager.service-f6vMrt
drwx- 3 root root 60 Sep 1 22:54 systemd-private-5b3c76ac0c3d443db5126f21f584e22b-systemd-logind.service-9WvMrt
```

Why can temporary directories be relevant during a cybersecurity investigation?
/tmp is relevant because it is a temporary, commonly writable location where programs may create temporary files. During a security investigation, suspicious scripts, downloaded files, extracted archives, or malware-related artifacts may be found there, so investigators should examine it carefully.

Exercise 14 – Explore the `/proc` Filesystem

Investigate:

/proc

...

Examine:

cat /proc/cpuinfo

cat /proc/meminfo

...

Then identify your current shell PID:

echo \$\$

...

Check whether a corresponding directory exists under `/proc`.

```
(prathiksha@kali)~$ cat /proc/cpuinfo
processor       : 0
vendor_id      : GenuineIntel
cpu family     : 6
model          : 140
model name     : 11th Gen Intel(R) Core(TM) i3-1115G4 @ 3.00GHz
stepping       : 1
microcode      : 0xffffffff
cpu MHz        : 2995.196
cache size     : 6144 KB
physical id    : 0
siblings       : 1
core id        : 0
cpu cores      : 1
apicid         : 0

(prathiksha@kali)~$ cat /proc/meminfo
MemTotal:      2016864 kB
MemFree:       199288 kB
MemAvailable:  1110872 kB
Buffers:       30864 kB
Cached:        1014448 kB
SwapCached:    4 kB
Active:        945940 kB
Inactive:      654824 kB
Active(anon):  478780 kB
Inactive(anon): 92176 kB
Active(file):  467160 kB
Inactive(file): 562648 kB
Unevictable:   112 kB
Mlocked:       112 kB
SwapTotal:     1127420 kB

(prathiksha@kali)~$ echo $$
2179

(prathiksha@kali)~$ ls /proc/$$
arch_status  cgroup      coredump_filter  e
attr         clear_refs  cpu_resctrl_groups  f
autogroup    cmdline    cwd               g
auxv         comm       environ
```

Answer:

1. What information does `/proc/cpuinfo` provide?

/proc/cpuinfo — shows details about the CPU(s): model, speed, cache size, and supported features. Since I'm in a VM, it only shows 1 core and flags it as running under a hypervisor.

2. What information does `/proc/meminfo` provide?

`/proc/meminfo` — shows live memory stats: how much RAM is total, free, and available, plus swap usage.

3. Why do directories with numbers appear inside `/proc`?

Numbered folders in `/proc` — each one is a running process's PID. My shell (PID 2179) has its own folder with live info like its command line, open files, and memory usage.

4. Is `/proc` an ordinary disk directory?

No — it's built by the kernel on the fly, purely in memory. Nothing here is actually saved to disk, and folders disappear the moment their process ends.

5. Why could `/proc` be useful during security investigation?

It lets you see exactly what's running right now, including a process's real command and open files/connections. That's useful for spotting something suspicious even if it's disguised or hidden from normal tools like `ps`.

Exercise 15 – Explore `/dev`

`/dev`

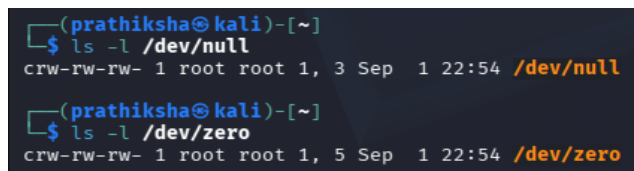
...

Examine:

`ls -l /dev/null`

`ls -l /dev/zero`

...



```
(prathiksha@kali)-[~]
$ ls -l /dev/null
crw-rw-rw- 1 root root 1, 3 Sep  1 22:54 /dev/null

(prathiksha@kali)-[~]
$ ls -l /dev/zero
crw-rw-rw- 1 root root 1, 5 Sep  1 22:54 /dev/zero
```

What does `/dev` represent?

It's where Linux exposes hardware and virtual devices as files — so programs can interact with things like disks, terminals, and these memory devices through normal file operations.

Are these ordinary user files?

No — the `c` at the start of the permissions marks them as character devices, not regular files. They don't hold data on disk; they're just interfaces to a kernel driver.

File type of `/dev/null`?

Character special device (major 1, minor 3).

Why does Linux expose devices this way?

It follows the "everything is a file" philosophy — instead of needing special device-specific code, any program can just open, read, or write to a device like it's a normal file. `/dev/null` discards anything written to it and returns EOF on read; `/dev/zero` returns endless zero bytes on read.

Exercise 16 – Filesystem Space Investigation

Determine:

Use appropriate combinations of:

`df`

`df -h`

`du`

`du -h`

`du -sh`

```

(prathiksha@kali)~$ df
Filesystem      1K-blocks      Used Available Use% Mounted on
udev            972960          0    972960   0% /dev
tmpfs           201688         1012    200676   1% /run
/dev/sda1       19353424 17499168    845824  96% /
tmpfs           1008432          4    1008428   1% /dev/shm
none            1024          0      1024   0% /run/credentials/systemd-journald.service
tmpfs           1008432        156    1008276   1% /tmp
none            1024          0      1024   0% /run/credentials/getty@tty1.service
tmpfs           201684        112    201572   1% /run/user/1000

(prathiksha@kali)~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            951M   0 951M   0% /dev
tmpfs           197M 1012K 196M   1% /run
/dev/sda1       19G   17G 826M  96% /
tmpfs           985M   4.0K 985M   1% /dev/shm
none            1.0M   0 1.0M   0% /run/credentials/systemd-journald.service
tmpfs           985M 176K 985M   1% /tmp
none            1.0M   0 1.0M   0% /run/credentials/getty@tty1.service
tmpfs           197M 112K 197M   1% /run/user/1000

(prathiksha@kali)~$ du -sh ~
248M    /home/prathiksha

(prathiksha@kali)~$ du -sh ~/cyber_lab
32K     /home/prathiksha/cyber_lab

```

Total filesystem capacity: 19G

Used space: 17G

Available space: 826M

Percentage utilization: 96%

Space occupied by home directory: 248M (/home/prathiksha)

Space occupied by cyber_lab: 32K (/home/prathiksha/cyber_lab)

Explain the difference between `df` and `du`.

df vs du

Think of storage like a house:

df tells you how full the whole house is. Here, / is 96% full, with 826 MB left.

du tells you what inside the house is using the space. Your home folder uses 248 MB, and cyber_lab uses only 32 KB.

So, df = overall disk usage, while du = folder/file usage.

Since your home folder is only 248 MB but the system has 17 GB used, most space is being used by system files, installed programs, /var, /usr, etc.

Exercise 17 – Cybersecurity Investigation Scenario

Assume that you are investigating a Linux machine after suspicious activity has been reported.

You are initially asked to examine only the filesystem.

Identify which of the following locations you would investigate and explain **why**:

/home

/etc

/var/log

/tmp

/proc

/dev

/root

For each selected location, state what type of security-relevant information you would expect to find.

/home – To look for suspicious files, scripts, downloads, hidden files, and user activity.

/etc – To check for changed settings, new users, sudo changes, or scheduled tasks.

/var/log – To find login activity, errors, and other clues about what happened.

/tmp – Attackers may use it to temporarily store malicious files or scripts.

/proc – Shows what is currently running on the system, which can help find suspicious processes.

/root – If accessible, it can show signs that an attacker gained root/admin access.

Low priority: /dev because it normally contains system device files and is less likely to contain useful evidence.